LAB 4: Data Cleaning and Transformation using Pandas

Procedure

Step 1:

Library Setup:
Pandas and NumPy libraries are imported.

Verification:
The versions are printed showing Pandas 2.3.3 and NumPy 2.3.5.

Purpose:
This confirms the environment is correctly configured.

```python
import pandas as pd
import numpy as np

# Confirming libraries are imported
print("Pandas version:", pd.__version__)
print("NumPy version:", np.__version__)
```

```
Pandas version: 2.3.3
NumPy version: 2.3.5
```

Figure 4.1

Step 2:

Dataset Loading:
The dataset 'data.csv' is loaded using pd.read_csv().

Confirmation:
A message 'Dataset loaded successfully' is printed.

Preview:
The first 5 rows are displayed showing columns like house age, distance to MRT, and house price.

```python
# Load dataset directly from existing CSV file
df = pd.read_csv('data.csv')

print("Dataset loaded successfully.\n")

# Optional Example: Load from Excel (requires openpyxl)
# df_excel = pd.read_excel('data.xlsx')

# Optional Example: Load from JSON
# df_json = pd.read_json('data.json')

print("First 5 rows of the dataset:")
display(df.head())
```

```
Dataset loaded successfully.

First 5 rows of the dataset:

   No  X1 transaction date  X2 house age  \
0   1             2012.917          32.0
1   2             2012.917          19.5
2   3             2013.583          13.3
3   4             2013.500          13.3
4   5             2012.833           5.0

   X3 distance to the nearest MRT station  X4 number of convenience stores  \
0                               84.87882                                10
1                              306.59470                                 9
2                              561.98450                                 5
3                              561.98450                                 5
4                              390.56840                                 5

   X5 latitude  X6 longitude  Y house price of unit area
0    24.98298    121.54024                        37.9
1    24.98034    121.53951                        42.2
2    24.98746    121.54391                        47.3
3    24.98746    121.54391                        54.8
4    24.97937    121.54245                        43.1
```

Figure 4.2

Step 3:

Data Inspection:
The dataset structure and values are displayed in detail.

Observation:
Multiple rows and columns are visible for analysis.

Purpose:
This step helps understand the dataset before processing.

```python
# Display first 5 rows
print("First 5 rows:")
display(df.head())

# Display last 5 rows
print("\nLast 5 rows:")
display(df.tail())

# Show dataset info (data types, non-null counts)
print("\nDataset Info:")
df.info()

# Show statistical summary for numeric columns
print("\nStatistical Summary:")
display(df.describe())

# Show shape of dataset (rows, columns)
print("\nDataset Shape:")
print(df.shape)
```

Figure 4.3

Step 4:

Column Selection:
Single and multiple columns such as house age and distance to MRT are selected.

Filtering:
Rows are filtered where house age > 20.

Conditional Filtering:
Additional conditions include price > 40 and distance < 500.

Output:
Filtered dataset is displayed.

```python
# Select a single column
house_age = df['X2 house age']

# Select multiple columns
subset = df[['X2 house age', 'X3 distance to the nearest MRT station', 'Y house price of unit area']]

# Filter rows using Boolean indexing (e.g., Houses older than 20 years)
older_houses = df[df['X2 house age'] > 20]

# Apply multiple conditions (e.g., Price > 40 and distance to MRT < 500)
premium_near_mrt = df[(df['Y house price of unit area'] > 40) & (df['X3 distance to the nearest MRT station'] < 500)]

print("Filtering complete.")
display(premium_near_mrt.head())
```

```
Filtering complete.

   No  X1 transaction date  X2 house age  \
1   2             2012.917          19.5
4   5             2012.833           5.0
7   8             2013.417          20.3
10 11             2013.083          34.8
11 12             2013.333           6.3

   X3 distance to the nearest MRT station  X4 number of convenience stores  \
1                               306.59470                                 9
4                               390.56840                                 5
7                               287.60250                                 6
10                              405.21340                                 1
11                               90.45606                                 9

   X5 latitude  X6 longitude  Y house price of unit area
1    24.98034    121.53951                        42.2
4    24.97937    121.54245                        43.1
7    24.98042    121.54228                        46.7
10   24.97349    121.53372                        41.4
11   24.97433    121.54310                        58.1
```

Figure 4.4

Step 5:

Sorting Data:
The dataset is sorted by house price in ascending order.

Multiple Sorting:
It also sorts by house age and price together.

Output:
Sorted data is displayed.

```python
# Sort by a single column (e.g., House price ascending)
df_sorted_price = df.sort_values(by='Y house price of unit area')

# Sort by multiple columns (e.g., Age ascending, then Price descending)
df_multi_sorted = df.sort_values(by=['X2 house age', 'Y house price of unit area'], ascending=[True, False])

print("Sorted by Age and Price:")
display(df_multi_sorted.head())
```

```
Sorted by Age and Price:

     No  X1 transaction date  X2 house age  \
166 167             2013.417           0.0
105 106             2012.833           0.0
16   17             2013.250           0.0
379 380             2013.333           0.0
258 259             2013.417           0.0

     X3 distance to the nearest MRT station  X4 number of convenience stores  \
166                               292.9978                                 6
105                               292.9978                                 6
16                                292.9978                                 6
379                               292.9978                                 6
258                               292.9978                                 6

     X5 latitude  X6 longitude  Y house price of unit area
166    24.97744    121.54458                        73.6
105    24.97744    121.54458                        71.0
16     24.97744    121.54458                        70.1
379    24.97744    121.54458                        69.7
258    24.97744    121.54458                        63.3
```

Figure 4.5

Step 6:

Missing Values Handling:
Missing values are identified using isnull().sum().

Filling Data:
Missing values are replaced using mean values of respective columns.

Result:
Clean dataset is confirmed.

```python
# Identify missing values
print("Missing values per column:")
print(df.isnull().sum())

# Fill missing values
# 1. Fill numeric 'X2 house age' with mean if there were any missing
df['X2 house age'] = df['X2 house age'].fillna(df['X2 house age'].mean())

# 2. Fill numeric 'X3 distance to the nearest MRT station' with mean
df['X3 distance to the nearest MRT station'] = df['X3 distance to the nearest MRT station'].fillna(df['X3 distance to the nearest MRT station'].mean())

print("\nMissing values after processing:")
print(df.isnull().sum())
```

```
Missing values per column:
No                                      0
X1 transaction date                     0
X2 house age                            0
X3 distance to the nearest MRT station  0
X4 number of convenience stores         0
X5 latitude                             0
X6 longitude                            0
Y house price of unit area              0
dtype: int64

Missing values after processing:
No                                      0
X1 transaction date                     0
X2 house age                            0
X3 distance to the nearest MRT station  0
X4 number of convenience stores         0
X5 latitude                             0
X6 longitude                            0
Y house price of unit area              0
dtype: int64
```

Figure 4.6

Step 7:

Feature Engineering:
A new column PricePerYear is created using price divided by house age.

Column Cleanup:
Unnecessary column No is removed.

Renaming:
Columns are renamed to more readable names like TransactionDate and DistanceToMRT.

Output:
Transformed dataset is displayed.

```python
# Add new column: PricePerYear (Price / House Age)
# Adding a small epsilon to avoid division by zero if HouseAge is 0
df['PricePerYear'] = df['Y house price of unit area'] / (df['X2 house age'] + 0.1)

# Remove columns (e.g., 'No' column is just an index and less useful)
df = df.drop(columns=['No'])

# Rename columns to be more readable
df = df.rename(columns={
    'X1 transaction date': 'TransactionDate',
    'X2 house age': 'HouseAge',
    'X3 distance to the nearest MRT station': 'DistanceToMRT',
    'X4 number of convenience stores': 'ConvenienceStores',
    'X5 latitude': 'Latitude',
    'X6 longitude': 'Longitude',
    'Y house price of unit area': 'UnitPriceArea'
})

print("Transformation complete.")
display(df.head())
```

```
Transformation complete.

   TransactionDate  HouseAge  DistanceToMRT  ConvenienceStores  Latitude  \
0         2012.917      32.0       84.87882                 10  24.98298
1         2012.917      19.5      306.59470                  9  24.98034
2         2013.583      13.3      561.98450                  5  24.98746
3         2013.500      13.3      561.98450                  5  24.98746
4         2012.833       5.0      390.56840                  5  24.97937

   Longitude  UnitPriceArea  PricePerYear
0  121.54024           37.9      1.180685
1  121.53951           42.2      2.153061
2  121.54391           47.3      3.529851
3  121.54391           54.8      4.089552
4  121.54245           43.1      8.450980
```

Figure 4.7

Step 8:

Exporting Data:
The cleaned dataset is saved as 'real_estate_cleaned.csv'.

Confirmation:
A message confirms successful file saving.

```python
# Export cleaned dataset to CSV file
df.to_csv('real_estate_cleaned.csv', index=False)

print("Cleaned dataset saved as 'real_estate_cleaned.csv'.")
```

```
Cleaned dataset saved as 'real_estate_cleaned.csv'.
```

Figure 4.8